



Tecnológico de Monterrey

Ian Armando Borde Escobar – A00846007

Reflexión Evidencia 1

Importancia del ordenamiento y la búsqueda en logs

Los logs contienen información importante sobre eventos de un sistema, como accesos no autorizados, errores, cambios en archivos y problemas de seguridad. Cuando los registros llegan desordenados, resulta difícil analizar qué ocurrió primero y establecer una secuencia de eventos.

Ordenar los registros por fecha y hora permite consultar la información cronológicamente. Esto facilita detectar patrones, investigar incidentes y conocer el orden en que ocurrieron los eventos. También es importante poder buscar un rango específico de fechas, porque revisar manualmente miles de registros sería lento y propenso a errores.

En este proyecto los archivos contienen 6818 registros. Por esta cantidad de datos, fue necesario utilizar algoritmos de ordenamiento y una búsqueda binaria para mejorar la eficiencia del programa.

Implementación realizada

Primero se implementó la lectura de los archivos log607-1.txt y log607-2.txt. Cada línea se conserva completa para poder escribirla nuevamente en los archivos de salida. Además, la fecha se convierte a un formato comparable:

AAAA-MM-DD HH:MM

Este formato permite comparar los timestamps como cadenas y mantener el orden cronológico.

Después se implementaron siete algoritmos de ordenamiento: Swap Sort, Selection Sort, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort y Shell Sort. El usuario puede elegir el archivo y el algoritmo desde el menú principal.

También se agregó la medición del tiempo utilizando microsegundos. Antes de cada corrida el usuario escribe una predicción de velocidad y una justificación. Después de ejecutar el algoritmo se muestra el tiempo real, la complejidad teórica y se registra si la predicción coincidió con el resultado.

Para la búsqueda por rango se implementaron dos búsquedas binarias. La primera encuentra la posición inicial mayor o igual que el límite inferior. La segunda encuentra la primera posición mayor que el límite superior. De esta forma el rango es inclusivo y se conservan todos los registros cuyos timestamps estén dentro del intervalo.

Cuando existen timestamps duplicados, cada registro se conserva individualmente. No se eliminan registros repetidos ni se agregan copias adicionales. Esta decisión permite que la búsqueda represente fielmente el contenido original del log.

Comparación de algoritmos

Los resultados mostraron diferencias importantes entre los algoritmos y los tipos de archivo.

En el archivo desordenado log607-1.txt, Swap Sort obtuvo un tiempo de 2,979,463 microsegundos y Bubble Sort obtuvo 3,106,473 microsegundos. Estos resultados son consistentes con su complejidad cuadrática, ya que ambos realizan muchas comparaciones e intercambios.

Selection Sort obtuvo 570,955 microsegundos e Insertion Sort obtuvo 492,365 microsegundos en el archivo desordenado. Aunque también tienen complejidad $O(n^2)$ en el peor caso, su implementación y cantidad de intercambios produjeron tiempos menores que Swap Sort y Bubble Sort.

Los algoritmos más eficientes para el archivo desordenado fueron Merge Sort, Quick Sort y Shell Sort. Merge Sort tardó 23,824 microsegundos, Quick Sort tardó 16,921 microsegundos y Shell Sort tardó 13,166 microsegundos.

En el archivo casi ordenado log607-2.txt, Insertion Sort tuvo un comportamiento muy favorable, con un tiempo de 1,920 microsegundos. Esto coincide con su mejor caso $O(n)$, porque cuando los datos ya están parcialmente ordenados necesita realizar pocos desplazamientos.

Bubble Sort también mejoró considerablemente con el archivo casi ordenado y obtuvo 9,303 microsegundos. Shell Sort tardó 7,141 microsegundos y Merge Sort tardó 19,207 microsegundos.

Un resultado inesperado fue Quick Sort con log607-2.txt. Aunque se predijo que sería rápido, tardó 1,376,506 microsegundos, mucho más que con el archivo desordenado. Esto puede explicarse por la forma en que se selecciona el pivote. Al utilizar el último elemento como pivote, un archivo casi ordenado puede provocar particiones poco equilibradas y acercar el comportamiento al peor caso $O(n^2)$.

Selection Sort tuvo tiempos muy similares en ambos archivos: 570,955 microsegundos en el archivo desordenado y 570,709 microsegundos en el casi ordenado. Esto tiene sentido porque Selection Sort realiza prácticamente la misma cantidad de comparaciones sin importar qué tan ordenados estén los datos.

Las predicciones no siempre coincidieron con los resultados. Por ejemplo, se predijo que Merge Sort, Quick Sort y Shell Sort serían lentos en el archivo desordenado, pero sus tiempos fueron relativamente bajos. También se predijo que Quick Sort sería rápido con el archivo casi ordenado, pero ocurrió lo contrario. Esto demuestra que la complejidad teórica sirve como referencia, pero el resultado real también depende de la implementación y de la distribución de los datos.

Uso crítico de inteligencia artificial

Utilicé inteligencia artificial como apoyo para resolver dudas específicas y revisar código propio. Algunos prompts utilizados fueron:

“¿Cómo puedo adaptar los algoritmos de ordenamiento de mi Actividad 1.5 para ordenar registros de logs por fecha y hora?”

“Revisa mi Ordenamientos.cpp y dime qué errores tiene al trabajar con `vector<Registro>`.”

“¿Cómo puedo implementar una búsqueda binaria por rango que incluya correctamente timestamps duplicados?”

La IA ayudó a explicar la diferencia entre archivos .hpp y .cpp, revisar errores de compilación, adaptar comparaciones para utilizar el campo timestamp y organizar el flujo del programa. Sin embargo, el código se compiló y probó manualmente antes de considerarlo correcto.

Durante la revisión se detectó un error en una versión de Selection Sort. La condición utilizaba una comparación incorrecta con la posición 1, cuando debía comparar la posición del elemento mínimo con la posición actual `i`. La condición correcta es `menor !=`

i. Si se dejaba la condición incorrecta, el algoritmo podía realizar intercambios equivocados y no ordenar correctamente todos los registros.

También se detectó que una primera organización solo consideraba cinco algoritmos, aunque la Actividad 1.5 contenía siete. Después de revisar nuevamente el código de la actividad anterior, se agregaron Swap Sort y Shell Sort.

Conclusión

La actividad permitió observar que no existe un algoritmo que sea el mejor para todos los casos. Los algoritmos cuadráticos pueden ser sencillos de implementar, pero resultan más lentos con archivos desordenados. Insertion Sort es una buena opción cuando los datos están casi ordenados. Merge Sort ofrece un comportamiento más estable, mientras que Quick Sort puede ser muy eficiente o acercarse a $O(n^2)$, dependiendo de la selección del pivote.

La búsqueda binaria permitió obtener rangos sin recorrer todos los registros desde el principio. El uso de límites inclusivos y de una búsqueda para la primera posición mayor que el límite final permitió manejar correctamente timestamps duplicados.

Finalmente, las mediciones ayudaron a relacionar la complejidad teórica con el comportamiento real. También demostraron la importancia de probar los algoritmos con diferentes tipos de datos y de revisar críticamente cualquier sugerencia proporcionada por una herramienta de inteligencia artificial.